AI-Powered Financial Fraud Detection System

Contents

1. Introduction
2. Objective
3. Dataset Used
4. Methodology - Step by Step Explanation
5. Results and Conclusion

1. Introduction

Financial fraud is one of the most damaging problems faced by banks, payment processors and e-commerce platforms. Every day millions of card payments, transfers and withdrawals are processed, and a very small fraction of them are fraudulent. Detecting that fraction quickly matters: an undetected fraudulent transaction results in a direct financial loss and a loss of customer trust, while an over-aggressive detection system blocks genuine customers and creates operational cost through manual review.

Traditional rule-based systems (for example, "flag every transaction above $5,000" or "flag every foreign transaction") are easy to understand but rigid. Fraudsters adapt quickly, rules multiply until they become unmaintainable, and static thresholds cannot capture the interaction between several weak signals — a moderate amount, at an unusual hour, on an unusual merchant category, through an unusual payment method.

This project builds a complete, working machine learning system that learns those patterns from historical transaction data instead of hard-coding them. It is not only a model in a notebook: it is a deployable application with a trained model, a REST API, a persistent prediction history and an interactive web interface, so that a non-technical analyst can score a transaction, upload a batch of transactions, review past decisions and retrain the model on new data.

1.1 Scope of the System

- A supervised classification model that outputs a fraud probability for any transaction.
- A REST API (FastAPI) exposing prediction, batch prediction, history, statistics and training endpoints.
- A relational store (SQLite via SQLAlchemy) that persists every prediction made, so results are auditable.
- An interactive dashboard (Streamlit) with six pages: Dashboard, Single Prediction, Batch Prediction, History, Train Model and Metrics.
- A reproducible synthetic dataset generator so the system can be demonstrated without access to confidential banking data.

2. Objective

The overall objective is to design, train, evaluate and deploy an interpretable machine learning system that identifies potentially fraudulent financial transactions in real time.

2.1 Primary Objectives

- Build a binary classifier that predicts whether a transaction is fraudulent (1) or legitimate (0), together with a calibrated probability score.
- Handle severe class imbalance, since fraudulent transactions represent only about 3% of all records.
- Maximise recall on the fraud class — it is far costlier to miss a real fraud than to review a false alarm.
- Keep the model interpretable, so that every decision can be explained to a compliance officer or auditor.

2.2 Secondary Objectives

- Expose the model through a documented REST API so any client application can consume it.
- Persist every prediction with its inputs, probability and timestamp for auditing and monitoring.
- Support batch scoring of a CSV file so an analyst can review a whole day of transactions at once.
- Allow retraining from the user interface when new labelled data becomes available.
- Present model quality transparently through metrics, a confusion matrix, an ROC curve and feature importance.

2.3 Why Logistic Regression

Logistic Regression was chosen deliberately over more complex models such as gradient boosting or neural networks:

- Interpretability — each feature has a single coefficient, so the direction and strength of every signal is directly readable. This is a regulatory requirement in financial services.
- Probability output — it produces well-behaved probabilities that can be thresholded to trade recall against precision according to business appetite.
- Speed — training takes seconds and inference is a single dot product, so the API responds in milliseconds.

- Robustness on tabular data with a modest number of features, especially when combined with proper scaling and resampling.

3. Dataset Used

Real transaction data is confidential and cannot be redistributed, so the project ships with a synthetic dataset generated by data/generate_dataset.py. The generator was designed to reproduce the statistical characteristics of real payment data: a low fraud base rate, a heavy-tailed amount distribution, and a fraud signal that is genuinely learnable but not trivially separable.

3.1 Dataset Overview

Property	Value
File	data/transactions.csv
Total records	12,000
Fraudulent records	413 (3.44%)
Legitimate records	11,587
Training set	9,000 rows (75%)
Test set	3,000 rows (25%), 103 fraudulent
Split strategy	Stratified, random_state fixed for reproducibility
Target variable	is_fraud (0 = legitimate, 1 = fraud)

3.2 Feature Dictionary

Column	Type	Description
transaction_id	Identifier	Unique reference such as TXN000001; excluded from modelling.
amount	Numeric	Transaction value in currency units; heavy right tail.
transaction_type	Categorical	purchase, withdrawal, transfer, payment.

merchant_category	Categorical	grocery, electronics, travel, online_services, entertainment, etc.
customer_age	Numeric	Age of the account holder in years.
payment_method	Categorical	card, wallet, bank_transfer, upi.
transaction_hour	Numeric (0-23)	Hour of day the transaction was initiated.
is_fraud	Target	1 if the transaction was confirmed fraudulent, otherwise 0.

3.3 Fraud Signal Embedded in the Data

The generator injects a multi-factor, probabilistic fraud signal rather than a single deterministic rule, so the classifier has to combine weak evidence:

- Higher transaction amounts raise fraud risk non-linearly (captured later by a log transform).
- Night-time transactions (roughly 00:00–05:00) carry elevated risk.
- Certain merchant categories, notably online_services, are disproportionately targeted.
- Transfers are riskier than ordinary in-store purchases.
- Random noise is added so that no combination of features guarantees fraud — mirroring real-world ambiguity.

The system also accepts user-supplied data. Any CSV containing the same eight columns can be uploaded on the Train Model page and used to retrain the pipeline, so the project can be moved onto real data without code changes.

4. Methodology — Step by Step Explanation

This section walks through the complete pipeline, from raw CSV to a live prediction served in the browser. Each step corresponds to an actual module in the repository.

Step 1 — Data Generation and Ingestion

Module: data/generate_dataset.py, ml/preprocessing.py

The generator draws each column from a realistic distribution: amounts from a log-normal distribution, ages from a truncated normal, hours from a day-cycle weighted distribution, and categories from empirical frequency weights. A fraud probability is then computed per row from the risk factors described in Section 3.3 and a Bernoulli draw decides the final label. This produces a dataset with a 3.4% fraud rate that behaves like real data. Ingestion simply reads the CSV with pandas and passes it to validation.

Step 2 — Data Validation and Cleaning

Module: ml/preprocessing.py

- Schema validation — the loader checks that every required column exists and raises a clear error naming any missing column, so a malformed upload fails fast instead of producing a wrong model.
- Type normalisation — numeric columns are coerced with errors='coerce' so that stray text becomes NaN rather than crashing the pipeline; categorical columns are lower-cased and stripped of whitespace so "Card", "card " and "card" collapse into one category.
- Missing value handling — numeric columns are imputed with the median (robust to the heavy tail of amount) and categorical columns with the mode.
- Range clipping — transaction_hour is constrained to 0–23 and negative amounts are rejected.
- Duplicate removal — repeated transaction_id rows are dropped to avoid leaking the same record into both train and test sets.

Step 3 — Feature Engineering

Module: ml/feature_engineering.py

Raw columns alone under-represent the fraud pattern, so four derived features are added before modelling:

Engineered feature	Definition	Rationale

log_amount	log(1 + amount)	Compresses the heavy right tail so the linear model can use amount effectively.
is_night	1 if hour is between 0 and 5	Encodes the elevated night-time risk as a clean binary flag.
is_high_amount	1 if amount exceeds a high percentile	Captures the outlier-value effect that a linear term alone would miss.
amount_per_age	amount / (customer_age + 1)	Relative spending intensity — a large amount is more suspicious for a young account.

Feature engineering is implemented as a scikit-learn compatible transformer, so exactly the same transformations are applied at training time and at prediction time. This eliminates training/serving skew, the single most common source of silent failure in deployed ML systems.

Step 4 — Encoding and Scaling

Module: ml/pipeline.py

- A ColumnTransformer routes each column group to the right treatment.
- Numeric features (amount, log_amount, customer_age, transaction_hour, amount_per_age and the binary flags) pass through a StandardScaler, giving each a mean of 0 and a standard deviation of 1. This matters for Logistic Regression because the L2 penalty is scale-sensitive: without scaling, features with large ranges would be penalised unfairly and the coefficients would not be comparable.
- Categorical features (transaction_type, merchant_category, payment_method) pass through OneHotEncoder with handle_unknown='ignore', so a category never seen during training does not crash the API — it is simply encoded as all zeros.

Step 5 — Handling Class Imbalance with SMOTE

Module: ml/pipeline.py

With only 3.4% positives, a model that predicts "legitimate" for every transaction would already be 96.6% accurate and completely useless. SMOTE (Synthetic Minority Over-sampling Technique) fixes this by generating synthetic fraud examples: for a given minority sample it selects one of its k nearest minority neighbours and creates a new point at a random position along the line joining them. The result is a balanced training set with plausible, non-duplicated fraud examples.

Two implementation details are critical and are both handled in the code:

- SMOTE is placed inside an imblearn Pipeline, not applied to the dataset beforehand. This guarantees resampling happens only on the training fold during cross-validation, never on validation or test data — applying SMOTE before splitting would leak information and inflate the reported scores.
- SMOTE runs after scaling and encoding, because it interpolates in the numeric feature space and needs comparable distances between points.

Step 6 — Model Training

Modules: ml/pipeline.py, ml/train.py

The classifier is scikit-learn's LogisticRegression with L2 regularisation, the lbfgs solver, an increased max_iter for convergence and a regularisation strength of C = 0.5. Training runs the entire pipeline end to end: feature engineering, column transformation, SMOTE, then the fit. The complete fitted pipeline — not just the classifier — is serialised with joblib to ml/model.pkl, so the preprocessing logic travels with the weights and prediction is a single call.

Logistic Regression models the log-odds of fraud as a linear combination of the transformed features and squashes it through the sigmoid function to produce a probability between 0 and 1.

Step 7 — Evaluation

Module: ml/evaluate.py

The held-out test set is never touched by SMOTE and retains the original 3.4% fraud rate, so the reported numbers reflect real-world conditions. The evaluation suite reports accuracy, precision, recall, F1, ROC-AUC, the confusion matrix, the full ROC curve and the ranked feature coefficients. All of these are written to ml/metrics.json and rendered on the Metrics page of the interface.

Step 8 — Backend API

Modules: backend/main.py, backend/database.py, backend/models.py, backend/schemas.py, backend/crud.py

FastAPI loads the serialised pipeline once at startup and exposes the following endpoints. Pydantic schemas validate every request body, so an out-of-range hour or an unknown payment method returns a descriptive 422 error rather than a bad prediction.

Endpoint	Method	Purpose
/health	GET	Liveness check and model-loaded status.

/predict	POST	Score a single transaction; returns label, probability and risk band.
/predict/csv	POST	Score an uploaded CSV in bulk and return the annotated results.
/history	GET	Paginated list of all past predictions with filters.
/history/{id}	DELETE	Remove a single history record.
/stats	GET	Aggregated counts, fraud rate and distributions for the dashboard.
/train	POST	Retrain the pipeline on the bundled or an uploaded dataset.
/metrics	GET	Latest evaluation metrics, ROC curve and feature importance.

Every prediction is written to a SQLite database through SQLAlchemy, storing the input features, the predicted label, the probability and a UTC timestamp. This gives the system a complete audit trail.

Step 9 — Frontend Application

Modules: frontend/app.py, frontend/pages/*, frontend/components/*

Page	Function
Dashboard	Headline KPIs, fraud-rate trend and distribution charts drawn from /stats.
Single Prediction	A form for one transaction with a probability gauge and risk band.
Batch Prediction	CSV upload, in-browser results table and a download of the scored file.
History	Searchable, filterable log of every past prediction with per-row deletion.
Train Model	Retrain on the bundled dataset or on a user-uploaded labelled CSV.
Metrics	Confusion matrix, ROC curve and ranked feature importance for the current model.

The frontend never touches the model directly; it communicates only through the HTTP API via a thin client in frontend/components/api.py. This separation means the model can be scaled, replaced or moved to another host without changing a line of interface code.

5. Results and Conclusion

5.1 Model Performance

Measured on the held-out test set of 3,000 transactions containing 103 genuine fraud cases:

Metric	Score	Interpretation
Accuracy	0.8797	Share of all transactions classified correctly.
Precision (fraud)	0.2041	Of the transactions flagged as fraud, this fraction really were fraud.
Recall (fraud)	0.8641	Of all actual frauds, this fraction was successfully caught.
F1-score (fraud)	0.3302	Harmonic mean of precision and recall.
ROC-AUC	0.9344	Probability the model ranks a random fraud above a random legitimate transaction.

5.2 Confusion Matrix

	Predicted: Legitimate	Predicted: Fraud
Actual: Legitimate	2550 (True Negatives)	347 (False Positives)
Actual: Fraud	14 (False Negatives)	89 (True Positives)

The model catches 89 of the 103 frauds present in the test set and misses only 14. It raises 347 false alarms out of 2,897 legitimate transactions.

5.3 Reading the Precision–Recall Trade-off

The headline tension in these results is high recall (86.41%) alongside low precision (20.41%). This is a deliberate consequence of the design, not a defect:

- Missing a fraud (a false negative) means an unrecovered financial loss plus a damaged customer relationship. The model produces only 14 of these.

- A false positive means a transaction is sent for review. It costs a few minutes of analyst time. At a 3.4% base rate, even a very good model produces many of these in absolute terms simply because legitimate transactions vastly outnumber fraudulent ones.
- ROC-AUC of 0.9344 is the metric to judge the model by, because it is independent of the decision threshold. It says the model ranks a random fraud above a random legitimate transaction about 93% of the time — strong discriminative power.
- Because the API returns the probability, not just the label, the operating threshold can be raised at any time to trade recall for precision without retraining. Raising it from 0.5 to, say, 0.8 sharply reduces the review queue at the cost of a few missed frauds.

5.4 Feature Importance

Because the model is linear, its coefficients are directly interpretable. A positive coefficient increases the log-odds of fraud; a negative one decreases it. The ten strongest signals learned by the model are:

Rank	Feature	Coefficient	Effect
1	log_amount	1.707	Increases fraud risk
2	is_night	1.675	Increases fraud risk
3	merchant_category_online_services	0.658	Increases fraud risk
4	transaction_type_transfer	0.608	Increases fraud risk
5	amount	0.605	Increases fraud risk
6	merchant_category_grocery	-0.567	Reduces fraud risk
7	transaction_type_withdrawal	0.560	Increases fraud risk
8	merchant_category_fuel	-0.555	Reduces fraud risk
9	transaction_type_purchase	-0.536	Reduces fraud risk
10	merchant_category_electronics	0.509	Increases fraud risk

The model independently rediscovered the risk structure that was built into the data: transaction size (log_amount) and night-time activity (is_night) dominate, followed by online-services merchants and transfer-type transactions. Grocery purchases push the prediction towards

legitimate. This alignment between learned coefficients and domain intuition is exactly the kind of evidence a compliance reviewer needs.

5.5 Conclusion

The project delivers a complete, working fraud detection system rather than an isolated model. Every stage of the machine learning lifecycle is implemented and reproducible: data generation, validation, feature engineering, imbalance correction, training, evaluation, serving and monitoring.

- Strong discrimination — a ROC-AUC of 0.9344 on genuinely imbalanced held-out data.
- High fraud capture — 86.41% of frauds are detected, which is the outcome the business actually cares about.
- Correct handling of imbalance — SMOTE applied inside the pipeline, so no information leaks into evaluation.
- Full interpretability — every decision traces back to a small set of readable coefficients.
- Production-shaped architecture — a serialised end-to-end pipeline, a validated REST API, a persistent audit log and a decoupled interface.

5.6 Limitations

- The dataset is synthetic. Real fraud patterns are richer, involve sequences of transactions across time, and drift as fraudsters adapt.
- Precision is low at the default 0.5 threshold, producing a substantial manual review queue.
- The model treats each transaction independently; it has no notion of customer history, velocity or device fingerprint.
- SQLite suits a single-instance deployment but is not appropriate for high-concurrency production traffic.

5.7 Future Work

- Add behavioural and velocity features: transactions per hour per customer, deviation from the customer's typical amount, distance from the usual location.
- Benchmark against Random Forest, XGBoost and LightGBM, and use SHAP values to retain explainability if a non-linear model wins.
- Tune the decision threshold against an explicit cost matrix instead of defaulting to 0.5.
- Add drift monitoring and scheduled retraining as new labelled outcomes arrive.

- Migrate the store to PostgreSQL and containerise the API for horizontal scaling.